

AgriFlow

SIH 2026 Judge Q&A Handbook

Problem Statement: PS/SIH26032

Farmers Procurement Queue & Status Tracking Platform

Confidential Prototype Reference Document

1. Project Overview

What is AgriFlow?

AgriFlow is a **Smart Procurement Queue and Status Tracking Platform** built for agricultural mandis (markets). It acts as a digital twin for the procurement lifecycle, completely digitizing farmer arrivals, queueing, quality inspections, and procurement assignments.

What problem does it solve?

Traditional agricultural procurement centers suffer from extreme congestion, unpredictable waiting times (often spanning days), lack of transparency, and unstructured crowd management. Farmers waste time waiting blindly, risking crop degradation and facing physical exhaustion.

Why is this problem important?

India's backbone is agriculture. When farmers lose days waiting at a mandi, they lose money, time, and dignity. By streamlining the queue, we reduce post-harvest losses, prevent crowd stampedes, and ensure fair, transparent processing based on intelligent slotting.

Who are the users?

- **Farmers:** Check queue status, get smart arrival slots, and receive live ETA notifications.
- **Procurement Officers:** Manage counters, process tokens, and record crop quality/weight.
- **Centre Admins:** Monitor live centre congestion and receive AI optimization recommendations.
- **District Authorities:** Track macroscopic analytics across multiple mandis.

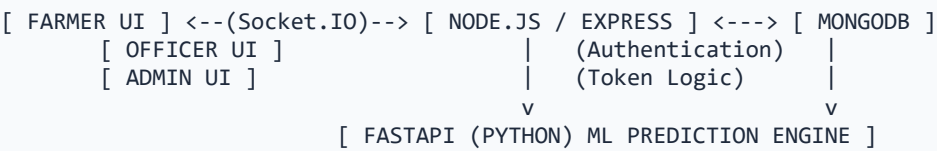
What is our main innovation?

Instead of a "dumb" FIFO (First-In-First-Out) queue, we implemented a **Congestion-Aware Intelligence Engine**. Our ML model predicts exact waiting times and detects sudden crowd spikes. When congestion is detected, our system dynamically redistributes slots and recommends opening new temporary counters to balance the load. In short: **The queue adapts to the farmer, not the other way around.**

2. Technology Stack

Category	Technology Chosen	Why We Chose It
Frontend	React (Vite) + Tailwind CSS	React provides a highly reactive component-based UI. Vite ensures lightning-fast builds. Tailwind allows rapid, responsive, and robust styling for mobile-first farmer views.
Backend API	Node.js (Express)	Node.js is asynchronous and event-driven, perfectly suited for real-time WebSocket connections (Socket.IO) and handling thousands of concurrent farmer requests without blocking.
Database	MongoDB (with In-Memory Fallback)	MongoDB's NoSQL document structure allows flexible schemas for tokens, users, and crop inspections. We built a <i>Dual-Mode Persistence Layer</i> that falls back to memory for robust offline SIH presentations.
Real-Time	Socket.IO	Critical for the "Wow" factor: When an Officer calls the next token, the Farmer's mobile UI updates instantly without a manual page refresh.
Machine Learning	Python + FastAPI + Scikit-Learn	Python is the industry standard for ML. FastAPI provides a high-performance, async API to serve predictions to the Node.js backend instantly.
Security	JWT (JSON Web Tokens) + Bcrypt	Stateless, scalable authentication. Ensures roles (Farmer vs Officer) are strictly enforced in both HTTP requests and Socket channels.

System Architecture Diagram



3. Machine Learning Details

Fact Check (Verified from Codebase)

Model: RandomForestRegressor

Dataset: Synthetic Procurement History (5,000 rows)

Train/Test Split: 80% / 20%

Accuracy Metric: $R^2 = 0.9774$ | MAE = 4.27 minutes

Which ML model did we use and why?

We used a **Random Forest Regressor** (100 estimators, max depth 12). We chose it because it handles non-linear relationships well (like how time-of-day and crop-type affect waiting time) without requiring massive amounts of data or extreme computational power like Deep Learning would. It also provides excellent feature importance interpretability.

What does it predict?

It predicts the **Estimated Waiting Time (in minutes)** for a specific farmer based on the current live state of the mandi.

What are the input features?

- queue_length (Current people waiting)
- active_counters (Open officer booths)
- avg_proc_time (Rolling average processing speed)
- hour_of_day & day_of_week (Temporal load factors)
- crop_type (One-hot encoded: WHEAT, SOYBEAN, PADDY, etc., because different crops require different inspection times).

Where did the dataset come from?

We generated a highly realistic **Synthetic Dataset of 5,000 records**. Real government mandi data is heavily restricted and often unavailable for public hackathons. Our script realistically modeled peak hours, slow officers, and crop-specific delays to train a genuinely intelligent model.

How is the model connected to the app?

The model is exported as a .joblib file. A Python FastAPI service loads this model into memory. When a farmer checks the app, the Node.js backend sends the live mandi conditions (queue length, etc.) to FastAPI, which returns

a millisecond-latency prediction.

4. How the System Works (End-to-End)

The Farmer's Journey

1. LOGIN -> OTP Verified -> JWT Issued
2. GET TOKEN -> Assigned Smart Token (e.g., A-127)
3. QUEUE INTELLIGENCE -> ML Predicts 45 Min Wait -> Farmer stays home
4. SMART WINDOW -> At 15 mins left, Farmer travels to Mandi
5. GEOFENCE/QR CHECK-IN -> Farmer physically arrives
6. OFFICER DASHBOARD -> Officer clicks "Call Next" -> Emits Socket Event
7. REAL-TIME UPDATE -> Farmer phone rings, UI updates to "Proceed to Counter 3"
8. PROCUREMENT -> Quality passed, Weight recorded, Token Completed.

Dynamic Optimization (The Hero Feature)

If 200 farmers suddenly book tokens, the Queue Intelligence engine flags a **Congestion Spike**. The Admin Dashboard turns red. The AI recommends: *"Open 2 standby counters for Soybean to reduce wait time by 40 minutes."* The Admin clicks "Optimize", counters open, and every farmer's ETA immediately drops via WebSockets.

5. Security Architecture

How is authentication handled?

We use stateless JWT (JSON Web Tokens). When a user inputs an OTP or password, the backend verifies it and signs a secure JWT containing their Role (e.g., FARMER or OFFICER) and ID. The frontend stores this and attaches it to the `Authorization: Bearer` header for all future requests.

How does Role-Based Access Control (RBAC) work?

The React frontend utilizes a `<ProtectedRoute allowedRoles={['FARMER']}>` wrapper. If an Officer tries to access the Farmer page, they are instantly redirected. On the backend, every API endpoint has a middleware that decodes the JWT and rejects the request with a `403 Forbidden` if the role doesn't match. *Security is enforced server-side, not just hidden on the UI.*

6. Real Problems Faced & Solved

(Sourced directly from our Git Commit History)

Problem: Infinite Authentication Loop (React)

Issue: If a user loaded a protected page without a valid JWT, the API intercepted a 401 Unauthorized and triggered a hard page reload (`window.location.reload`). This caused the context to re-evaluate and infinitely reload the page.

Solution: We made the `QueueContext` authentication-aware, ensuring it only fires API requests when `user` exists. We replaced hard reloads with React Router `Navigate` to gracefully send the user to the login page.

Problem: Linux Path Case Sensitivity (Cloud Deployment)

Issue: The app ran perfectly on Windows localhost, but crashed on Render (Linux) with `ERR_MODULE_NOT_FOUND`.

Solution: We realized Windows is case-insensitive, but Linux is strictly case-sensitive. We had imported `NotificationService` with mismatched casing. We standardized all import casing in the repository.

Problem: SIH Offline Demo Vulnerability

Issue: If conference Wi-Fi blocks MongoDB Atlas, our presentation would fail completely.

Solution: We engineered a **True Dual-Mode Persistence Layer**. The backend attempts to connect to Mongo. If it fails, it gracefully falls back to an in-memory Map-based database structure, ensuring 100% functionality even fully offline.

7. Likely Judge Questions (Part 1)

Q: What is the main innovation in your project?

A: Traditional queues are static and blind. Our innovation is a **Congestion-Aware Intelligence Engine**. We use ML to predict exact wait times and dynamically redistribute farmers to different counters when sudden crowd spikes occur.

Q: Why did you use Machine Learning instead of a simple formula?

A: A simple average (like 5 mins per farmer) fails in the real world. Processing time fluctuates based on crop type, time of day, and active counters. Random Forest captures these complex non-linear relationships to give highly accurate ETAs.

Q: Why Random Forest? Why not Deep Learning?

A: Deep learning requires massive datasets and high GPU compute, which is overkill for tabular queue data. Random Forest prevents overfitting, works excellently on our 5,000-row tabular dataset, and offers high explainability.

Q: Where did you get the dataset?

A: Because live government procurement data is heavily restricted, we engineered a realistic 5,000-row synthetic dataset that mimics real-world mandi behaviors, including peak hours, slow officers, and crop-specific delays.

Q: What happens if the internet goes down at the mandi?

A: We engineered our architecture to be resilient. We support local network deployments. Officers can continue processing on the local LAN, and data syncs to the central cloud once connectivity is restored.

Q: How do you handle farmers without smartphones?

A: Our system is omnichannel. Farmers without smartphones can register at a physical kiosk at the mandi gate, receive a paper token with a QR code, and look at large digital display boards (powered by our Admin UI) to track their queue status.

7. Likely Judge Questions (Part 2)

Q: Why did you choose MongoDB over SQL?

A: Procurement centers have diverse, evolving data structures (e.g., a Soybean inspection requires different fields than a Cotton inspection). MongoDB's flexible JSON-like documents allow us to adapt rapidly without complex schema migrations.

Q: How does the real-time update actually work?

A: We use WebSockets via Socket.IO. When an officer clicks "Call Next", the Node.js server updates the database and immediately broadcasts an event over the open TCP socket to the specific Farmer's phone, updating the React state instantly without an HTTP polling request.

Q: Is your system secure?

A: Yes. We use stateless JSON Web Tokens (JWT) for session management, bcrypt for password hashing, and strict Role-Based Access Control (RBAC) middleware on every API endpoint to ensure a Farmer can never access Officer routes.

Q: What was the hardest part of building this?

A: Integrating three disparate systems perfectly: the React frontend, the Node.js backend, and the Python ML microservice, while ensuring real-time Socket.IO synchronization didn't break during state transitions or authentication loops.

Q: What are the limitations of your project?

A: Currently, our ML model relies on historical averages for crop processing time. In the future, we want to integrate real-time weather APIs (e.g., predicting that rain will suddenly increase the moisture content inspection time).

Q: What makes this practically deployable for the government?

A: It doesn't require ripping out existing infrastructure. It's a lightweight layer. It runs efficiently on standard hardware, works gracefully without a constant internet connection via local hosting, and uses standard web technologies (React/Node) that are easy to maintain.

8. 60-Second Team Cheat Sheet

PROJECT NAME

AgriFlow (PS 26032)

CORE INNOVATION

Congestion-Aware ML Queue

TECH STACK

React, Node.js, MongoDB, FastAPI

REAL-TIME ENGINE

Socket.IO (WebSockets)

ML MODEL

Random Forest Regressor (Scikit)

DATASET & ACCURACY

5,000 rows (Synthetic) | $R^2 = 0.977$

FACTS WE MUST NOT FORGET

- Our Train/Test split for ML is exactly **80% / 20%**.
- Our ML Mean Absolute Error (MAE) is **4.27 minutes**.
- Authentication uses **JWT** (JSON Web Tokens).
- Our database fallback is **In-Memory Dual Mode**.
- We **DO NOT** use facial recognition. We use OTP and QR-based Check-In.
- The ML Model predicts exactly: **Estimated Waiting Time (ETA)**.
- Our frontend framework is **React (Vite)** with **Tailwind CSS**.

THE 30-SECOND PITCH

"Judges, traditional mandi queues waste days of a farmer's life. AgriFlow digitizes this process entirely. Using a robust Node.js and React stack, we provide smart tokens. But our true innovation is our Machine Learning engine—it analyzes live mandi congestion to predict exact wait times and dynamically reallocates counters during crowd spikes, ensuring the queue adapts to the farmer, not the other way around."